

Sistemas Distribuídos e Computação em Nuvem

Aula 2 — Modelos de arquitetura e sockets (TCP/UDP)

C1 · Fundamentos e comunicação distribuída · 13/08/2026

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Retomada — o que você fez em casa

 A aula começa consolidando a entrega da Aula 1.

- Sua **VM** (ou o Windows nativo) roda **Python** e **Git** sem erro?
- O repositório `sd-2026-2` está no GitHub com o primeiro commit?
- Conseguiu rodar `servidor_eco.py` / `cliente_eco.py` e **ver o erro** ao derrubar o servidor?

 Dúvida sobre a atividade de casa? **É agora o momento de perguntar.**

Objetivos desta aula

Ao final, você será capaz de:

1. **Diferenciar** os modelos **cliente-servidor** e **ponto a ponto (P2P)**.
2. **Explicar**, com suas palavras, a diferença prática entre **TCP** e **UDP**.
3. **Escrever** um cliente e um servidor que trocam mensagens por **socket**.

Conceito 1/3 — Cliente-servidor × P2P

Cliente-servidor

- Papéis **fixos**: um **pede** (cliente), o outro **responde** (servidor).
- Modelo **dominante** na web.
- Simples de gerenciar e proteger, **mas** concentra carga e é **ponto único de falha**.

Ponto a ponto (P2P)

- Todos têm o **mesmo papel** e trocam direto, **sem** servidor central.
- Ex.: **BitTorrent** (cada um baixa e envia).
- Distribui carga e não tem centro que caia, **mas** é bem mais difícil de **coordenar e proteger**.

💡 Cliente-servidor tem papéis fixos; em P2P todos fazem os dois papéis. A maioria do que você vai construir é **cliente-servidor**.

Conceito 2/3 — Socket: a ponta do cano

- Toda comunicação em rede passa, por baixo, por **sockets**.
- Um **socket** é a **ponta de uma conexão**, identificada por **IP + porta**: o **IP** diz *em qual máquina*; a **porta** diz *qual programa* dentro dela recebe.
- Por isso um mesmo servidor roda um **site na 443** e um **banco na 5432** sem confusão — cada serviço **escuta a sua porta**.

O **servidor** abre a porta e espera → `bind` · `listen` · `accept`

O **cliente** se conecta àquela porta → `connect` · e então os dois **trocam bytes**.

💡 Tudo que vem depois no curso — **gRPC, REST, filas** — roda sobre essa base.

Conceito 3/3 — TCP × UDP: garantia × velocidade

TCP — garantia

- **Cria conexão** antes de enviar.
- **Garante** entrega **e ordem**; **retransmite** o que se perde.
- Custa **tempo** e pacotes de controle.
- Use quando **não pode perder**: arquivo, chat, **transação bancária**.

UDP — velocidade

- **Sem conexão**, **não garante** nada.
- **Rápido e leve**; pode **perder** e **desordenar**.
- Sua aplicação lida com isso.
- Use quando **atraso é pior que perda**: **voz, vídeo ao vivo, jogo, telemetria**.

💡 Regra prática: **TCP** quando receber tudo, na ordem, importa mais que a pressa; **UDP** quando é melhor descartar um quadro atrasado do que travar esperando por ele.

Laboratório

Servidor de eco em TCP — passo a passo

Lab · Passo 1 — o servidor (servidor_eco.py)

```
import socket

HOST, PORT = "127.0.0.1", 5000          # localhost + porta escolhida

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  # TCP
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1) # reusar a porta
s.bind((HOST, PORT))                                # reserva o endereço
s.listen()                                           # passa a ESCUTAR
print(f"[servidor] ouvindo em {HOST}:{PORT}", flush=True)

conexao, endereco = s.accept()                    # ACEITA um cliente (bloqueia)
print(f"[servidor] cliente conectado: {endereco}", flush=True)
while True:
    dado = conexao.recv(1024)                     # RECEBE bytes
    if not dado:                                   # cliente fechou → sai
        break
    print(f"[servidor] recebi: {dado.decode()}", flush=True)
    conexao.sendall(dado)                          # ECO: devolve o mesmo
conexao.close()
```


Lab · Passo 2 — o cliente (cliente_eco.py)

```
import socket

HOST, PORT = "127.0.0.1", 5000

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  # TCP
s.connect((HOST, PORT))                               # CONECTA no servidor
print("Conectado. Digite mensagens (ou 'sair'):")
while True:
    msg = input("> ")
    if msg == "sair":
        break
    s.sendall(msg.encode())                            # ENVIA bytes
    eco = s.recv(1024)                                # RECEBE o eco
    print("eco:", eco.decode())
s.close()
```

💡 Note a simetria: o servidor faz `bind/listen/accept` e o cliente faz `connect` ; depois **os dois** usam `send / recv` .

Da demo (Aula 1) ao lab (Aula 2)

Os scripts têm o **mesmo esqueleto de socket**, mas propósitos diferentes:

Demo da Aula 1 — ver a falha

- Cliente manda **uma** mensagem fixa (`"oi"`).
- Servidor **finge processar** (`time.sleep(15)`) para você **derrubá-lo** no meio.
- Cliente tem `settimeout` + `try/except` só para **observar** o erro.

Lab da Aula 2 — o eco funcional

- Cliente **interativo** (`input`), envia **várias** mensagens até `sair`.
- Servidor em `while True` ecoa mensagem após mensagem (sem `sleep`).
- **Sem** timeout e **sem** tratamento de erro — o **caminho feliz**.

⚠ O `timeout` e o tratamento de falha **voltam na Aula 11 (resiliência)**. Por isso o **Passo 4** derruba o servidor: você reencontra a **falha parcial**, agora no **seu** eco.

Lab · Passo 3 — rodar (dois terminais)

Terminal 1 — servidor:

```
cd $HOME\Documents\sd-2026-2
python servidor_eco.py
# [servidor] ouvindo em 127.0.0.1:5000
```

Terminal 2 — cliente:

```
cd $HOME\Documents\sd-2026-2
python cliente_eco.py
# Conectado. Digite mensagens (ou 'sair'):
> ola mundo
eco: ola mundo
```

💡 No VS Code, abra **dois terminais** (menu Terminal → *Split Terminal*) e rode um em cada.

Lab · Passo 4 — o experimento

Com o **cliente conectado**, volte ao **Terminal 1** e **derrube o servidor** com `Ctrl + C`. Depois, no cliente, **envie outra mensagem**.

- O que acontece com o cliente?
- Ele **avisa** claramente que o servidor caiu, ou só **falha ao esperar**?

⚠ É a **falha parcial** da Aula 1, agora no seu código: do lado do cliente, "o servidor caiu" e "o servidor está lento" chegam do **mesmo jeito**. Guarde isso — na fase de resiliência vamos tratar com **tempo-limite + nova tentativa**.

Lab · Checkpoints & problemas comuns (Windows)

✓ O que você deve ver

- Servidor: ouvindo em `127.0.0.1:5000`.
- Cliente: cada `> mensagem` volta como `eco: mensagem`.
- Entrega: `servidor_eco.py` + `cliente_eco.py` **commitados**.

🔧 Se der erro

- `python` não encontrado → use `py`.
- `[WinError 10048]` (porta ocupada) → troque a porta (ex.: 5001) ou feche o outro processo.
- `[WinError 10054]` (conexão resetada) → é o **servidor que caiu** (o experimento!).
- Prompt do **Firewall** → como usamos `127.0.0.1`, normalmente nem aparece; se aparecer, permita em rede privada.

No seu trabalho — C1.A2

- **Sockets são o alicerce invisível** do trabalho: as interfaces **gRPC** (Aula 4) e **REST** (Aula 5) que você vai expor **rodam sobre eles**.
- Você **não os programa diretamente**, mas entender esta camada explica **por que uma chamada ao serviço pode ficar "pendurada"** — é o fundamento do **tempo-limite** que você vai adicionar na **resiliência** (Aula 11).

💡 É a base sobre a qual as duas interfaces do C1.A2 são construídas. Repositório: `sd-2026-2-kit-c1a2`.

Atividade para casa — UDP e comparação

1. **Reescreva** o eco em **UDP**: use `socket.SOCK_DGRAM`; **não há** `listen/accept/connect`. Troque `send` / `recv` por `sendto` / `recvfrom` (que já trazem o endereço do remetente).
2. **Meça** o tempo de ida e volta (*round-trip*) de **100 mensagens** em **TCP** e depois em **UDP**.
3. **Escreva 5 linhas** em `medicao.md` comparando os dois resultados que você **mediu**.
4. **Commit e push**.

📌 **Entregar até a próxima aula:** `eco_udp.py` + `medicao.md` com a comparação TCP × UDP.

💡 Dica de medição: use `time.perf_counter()` antes e depois do laço de 100 mensagens e divida pela quantidade.

◆ Foco ENADE

O que costuma cair:

- Modelo **cliente-servidor** comparado ao **P2P**.
- **Características de TCP e UDP** e o **critério** para escolher entre eles.
- Conceito de **socket, porta e endereçamento**.
- **Camadas do modelo TCP/IP** e encapsulamento.

Termos-chave: Cliente-servidor · P2P · Socket · Porta · TCP · UDP

💡 Um dos tópicos **mais cobrados**: espere cenários (streaming, transferência de arquivo, telemetria) que pedem a escolha **justificada** entre TCP e UDP.

Questão de autoavaliação (estilo ENADE)

Uma aplicação de **voz sobre IP** prioriza **baixa latência** e **tolera a perda ocasional** de pequenos trechos de áudio. Qual protocolo de transporte é o mais adequado e por quê?

- A) TCP, porque garante a entrega ordenada de todos os pacotes.
- B) TCP, porque realiza controle de fluxo e congestionamento.
- C) UDP, porque não retransmite pacotes perdidos, evitando o atraso acumulado.
- D) UDP, porque confirma o recebimento de cada pacote enviado.
- E) UDP, porque estabelece conexão prévia e reduz a latência.

Resolução — alternativa C

- Em **mídia de tempo real**, o **atraso é pior que a perda**: o **UDP não retransmite** e mantém o **fluxo fluido**.
- **D** e **E** descrevem o UDP **incorretamente** — ele **não** confirma recebimento nem **estabelece conexão**.
- **A** e **B** trazem garantias do TCP, que aqui **atrapalhariam** (retransmissão = atraso acumulado).

Fora da sala · Glossário

Para estudar

- **Coulouris**, cap. 4 — Comunicação entre processos.
- Documentação oficial do módulo **socket** do Python.
- **Refaça o desenho** do diagnóstico da Aula 1, agora com os **nomes corretos** (socket, porta, TCP...).

Glossário

- **Socket**: ponta de conexão (IP + porta).
- **Porta**: número que diz qual programa recebe.
- **TCP**: com conexão; garante entrega e ordem.
- **UDP**: sem conexão; rápido, sem garantias.
- **P2P**: todos com o mesmo papel.

Até a próxima aula 🚀

Entregue `eco_udp.py` + `medicao.md`. A Aula 3 começa revendo isto.

Próxima: Concorrência — um servidor para vários clientes (threads).

 **Voltar ao índice**
todas as aulas